

C++ Function Overloading

In C++, two functions can have the same name if the number and/or type of arguments passed is different.

These functions having the same name but different arguments are known as overloaded functions. For example:

```
// same name different arguments
int test() { }
int test(int a) { }
float test(double a) { }
int test(int a, double b) { }
```

Here, all 4 functions are overloaded functions.

Notice that the return types of all these 4 functions are not the same. Overloaded functions may or may not have different return types but they must have different arguments. For example,

```
// Error code
int test(int a) { }
double test(int b){ }
```

Here, both functions have the same name, the same type, and the same number of arguments. Hence, the compiler will throw an error.

Rules of Function Overloading in C++

There are certain rules to be followed while overloading a function in C++. Let us have a look at some of them:

1. The functions must have the same name
2. The functions must have different types of parameters.
3. The functions must have a different set of parameters.
4. The functions must have a different sequence of parameters.

Example 2:

```
#include <iostream>
using namespace std;

int plusFuncInt(int x, int y) {
```

```

    return x + y;
}

double plusFuncDouble(double x, double y) {
    return x + y;
}

int main() {
    int myNum1 = plusFuncInt(8, 5);
    double myNum2 = plusFuncDouble(4.3, 6.26);
    cout << "Int: " << myNum1 << "\n";
    cout << "Double: " << myNum2;
    return 0;
}

```

Example:

```
#include <iostream>
```

```
using namespace std;
```

```

    void print(int i) {

        cout << "integer value: " << i << endl;

    }

    void print(double f) {

        cout << "floating value: " << f << endl;

    }

    void print(char* c) {

        cout << "character value: " << c << endl;

    }

```

```

int main() {

    print(512);

    print(90.75);
}

```

```
    print("Podium");

    return 0;

}
```

Example:

```
#include <iostream>

using namespace std;

    int sum(int a,int b)

    {

        return a+b;

    }

    float sum(float a,float b, float c)

    {

        return a+b+c;

    }

int main(void)

{

    cout<<sum(2, 11)<<endl;

    cout<<sum(2.1, 10.12, 7.9);

    return 0;

}
```

- ***Type Conversion:***

Let's see a simple example.

```
#include<iostream>
using namespace std;
void fun(int);
```

```

void fun(float);
void fun(int i)
{
    std::cout << "Value of i is : " <<i<< std::endl;
}
void fun(float j)
{
    std::cout << "Value of j is : " <<j<< std::endl;
}
int main()
{
    fun(12);
    fun(1.2);
    return 0;
}

```

The above example shows an error "**call of overloaded 'fun(double)' is ambiguous**". The fun(12) will call the first function. The fun(1.2) calls the second function according to our prediction. But, this does not refer to any function as in C++, all the floating point constants are treated as double not as a float. If we replace float to double, the program works. Therefore, this is a type conversion from float to double.